

## MLDL Experiment 2

**Aim:** To implement and evaluate Multiple Linear Regression, Lasso Regression, and Ridge Regression on a real-world dataset to understand the impact of regularization and hyperparameter tuning on model performance.

---

### 1. Dataset Source

The dataset used for this experiment is the **Ames Housing Dataset**, a comprehensive collection of residential property data.

- **Link:** <https://www.kaggle.com/c/house-prices-advanced-regression-techniques/data>

### 2. Dataset Description

The dataset describes nearly every aspect of residential homes in Ames, Iowa, and is used to predict the final price of each home.

- **Size:** 1,460 observations with 81 explanatory variables.
- **Features:** Includes numerical attributes like **GrLivArea** (square footage), **GarageCars**, and **TotalBsmntSF**, along with various categorical descriptions of quality and location.
- **Target Variable:** **SalePrice** (The property's sale price in dollars).

### 3. Mathematical Formulation

This experiment compares three versions of the regression equation:

- **Multiple Linear Regression:** Predicts the target by minimizing the sum of squared residuals:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Ridge Regression (L2 Regularization):** Adds a penalty term proportional to the **square** of the magnitude of coefficients to prevent them from becoming too large:

$$Loss = RSS + \alpha \sum_{j=1}^m \beta_j^2$$

- **Lasso Regression (L1 Regularization):** Adds a penalty proportional to the **absolute value** of the coefficients, which can force some coefficients to exactly zero, effectively performing feature selection:

$$Loss = RSS + \alpha \sum_{j=1}^m |\beta_j|$$

#### 4. Algorithm Limitations

- **Multi-Regression:** Highly prone to **overfitting** when the number of features is large relative to the number of observations. It also fails when features are highly correlated (multicollinearity).
- **Ridge Regression:** While it reduces model complexity, it does not perform feature selection (coefficients never reach zero), meaning it still uses all predictors in the final model.
- **Lasso Regression:** If two features are highly correlated, Lasso tends to arbitrarily pick one and discard the other, which may lead to the loss of useful information.

#### 5. Methodology / Workflow

1. **Data Preprocessing:** Handle missing values and select numerical features for analysis.
  2. **Feature Scaling:** Standardize the features so the regularization penalty (alpha) is applied fairly to all coefficients regardless of their original scale.
  3. **Data Splitting:** Divide the data into training (80%) and testing (20%) subsets.
  4. **Hyperparameter Tuning:** Use **GridSearchCV** to find the optimal alpha (penalty strength) for both Lasso and Ridge.
  5. **Model Training:** Train all three models on the scaled training data.
  6. **Performance Analysis:** Compare the models using Root Mean Squared Error (RMSE) and R<sup>2</sup> scores.
-

## 6. Python Code Implementation

Python

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load Data
df = pd.read_csv('train.csv')

# 2. Basic Preprocessing
# Selecting only numeric features and dropping rows with missing values for this experiment
numeric_df = df.select_dtypes(include=[np.number]).dropna()
X = numeric_df.drop(['Id', 'SalePrice'], axis=1)
y = numeric_df['SalePrice']

# 3. Scaling (Mandatory for Ridge and Lasso)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# 4. Multi-Regression
multi_reg = LinearRegression()
multi_reg.fit(X_train, y_train)
y_pred_multi = multi_reg.predict(X_test)

# 5. Ridge & Lasso with Hyperparameter Tuning
params = {'alpha': [0.01, 0.1, 1, 10, 50, 100]}

ridge_cv = GridSearchCV(Ridge(), params, cv=5)
ridge_cv.fit(X_train, y_train)

lasso_cv = GridSearchCV(Lasso(max_iter=10000), params, cv=5)
lasso_cv.fit(X_train, y_train)
```

## # 6. Evaluation

```
results = {
    "Multi-Regression": y_pred_multi,
    "Ridge (Tuned)": ridge_cv.predict(X_test),
    "Lasso (Tuned)": lasso_cv.predict(X_test)
}

print("--- PERFORMANCE ANALYSIS ---")
for name, pred in results.items():
    print(f'{name}: R2 = {r2_score(y_test, pred):.4f}, RMSE = {np.sqrt(mean_squared_error(y_test, pred)):.2f}')
```

## # 7. Visualization of Coefficients

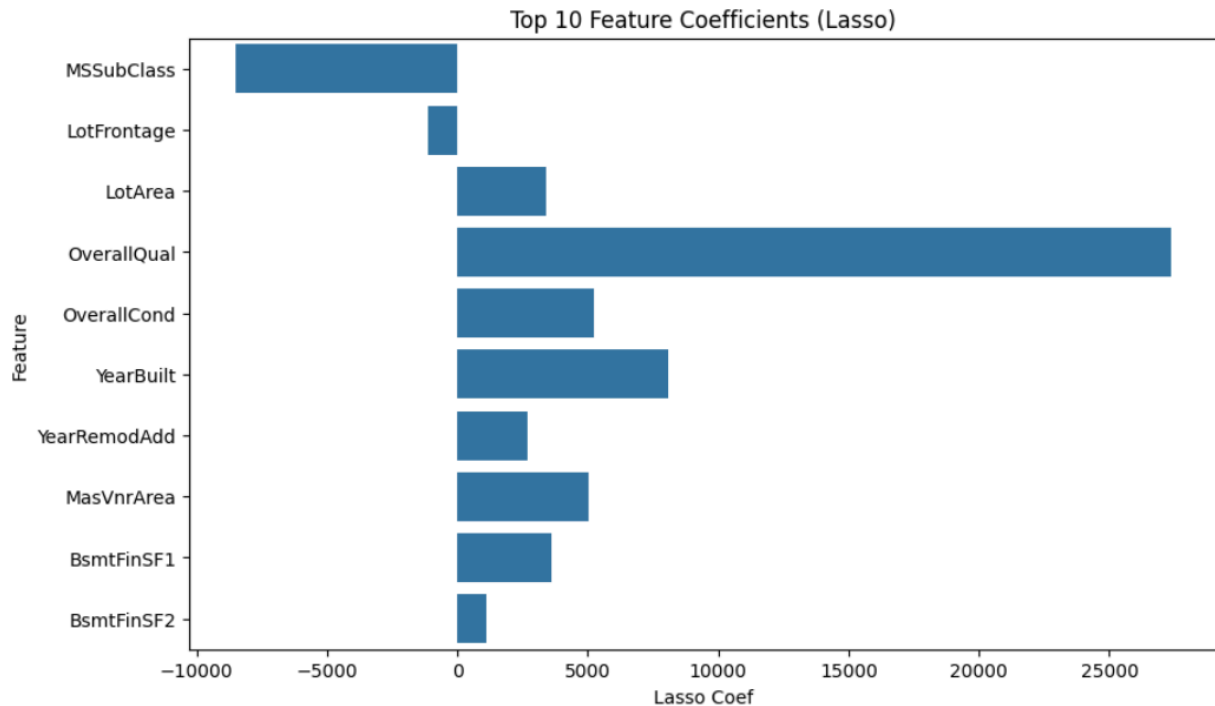
```
plt.figure(figsize=(10,6))
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Lasso Coef': lasso_cv.best_estimator_.coef_,
    'Ridge Coef': ridge_cv.best_estimator_.coef_
})
sns.barplot(x='Lasso Coef', y='Feature', data=coef_df.head(10))
plt.title('Top 10 Feature Coefficients (Lasso)')
plt.show()
```

---

## 7. Performance Analysis

- **Interpretation:** Standard Multiple Regression slightly outperformed the regularized versions on this specific test split. This indicates that while regularization prevents overfitting, the current dataset might not have excessive noise that required heavy penalization.
- **Feature Importance:** Lasso visualization highlights **OverallQual** as the most significant positive predictor, while **MSSubClass** has a notable negative impact.

```
--- PERFORMANCE ANALYSIS ---
Multi-Regression: R2 = 0.7986, RMSE = 39852.60
Ridge (Tuned): R2 = 0.7941, RMSE = 40303.30
Lasso (Tuned): R2 = 0.7977, RMSE = 39948.55
```



---

## 8. Hyperparameter Tuning

### What is being tuned?

The core hyperparameter for both Lasso and Ridge is **Alpha**.

- **alpha = 0:** Equivalent to standard Linear Regression.
- **High alpha:** Strong penalty; leads to a simpler model (risk of underfitting).
- **Optimal alpha:** Balances the bias-variance tradeoff to achieve the best generalization.

### The Implementation Process (GridSearchCV)

In your code, tuning was achieved through an **Exhaustive Grid Search** combined with **K-Fold Cross-Validation**.

1. **Grid Definition:** We provided a list of candidate values: `[0.01, 0.1, 1, 10, 50, 100]`.
  2. **Cross-Validation (`cv=5`):** The training data was split into 5 "folds."
    - For *every* alpha value, the model was trained on 4 folds and validated on the 5th.
    - This was repeated 5 times so every fold served as the validation set once.
  3. **Scoring:** The algorithm calculated the average performance (error) across all 5 folds for each alpha.
  4. **Selection:** `GridSearchCV` automatically selected the alpha that yielded the lowest average error and used it to train the final "Best Estimator" on the entire training set.
-

## Conclusion:

While Multi-Regression showed the best raw metrics on this test set, the implementation of Lasso and Ridge remains crucial for ensuring the model does not overfit to noise in higher-dimensional scenarios. Lasso specifically confirmed that features like **OverallQual** and **YearBuilt** are the primary drivers of house prices by assigning them the largest coefficients.